

INFORME DE BUGS

Proyecto

Testing de la plataforma de
Pinterest Lite (simulación)

QA-Tester

Fátima Paz

Introducción

El proyecto final tiene como objetivo recopilar y aplicar los conocimientos adquiridos durante el curso de QA Testing en ADA, simulando el trabajo de un equipo de Quality Assurance en un entorno de desarrollo ágil.

Para este proyecto se eligió como base de simulación la aplicación de Pinterest Lite, con el fin de ejecutar las pruebas necesarias que validan el comportamiento esperado del sistema en distintos escenarios.

Herramientas y procesos

Haciendo uso de la herramienta de GSuite, Notes, GitHub, Postman, se desarrollará el siguiente proyecto para realizar un testing a la plataforma Pinterest Lite desde la PC (versión escritorio) en Chrome con el fin de observar si ésta corre de la misma manera que se ha reportado entre los usuarios dentro de la app en un dispositivo móvil y si esta resulta interesante para los usuarios que deseen utilizar Pinterest en su ordenador.

Alcance del proyecto

Durante este ejercicio se realizaron pruebas funcionales enfocadas en validar diferentes flujos de la aplicación, incluyendo:

- Registro de usuario
 - Crear una cuenta/usuario en Pinterest
- Inicio de sesión
 - Entrar a Pinterest con credenciales válidas

- Visualización de contenido
 - Ver pines (feedback)
- Búsqueda de pines
 - Buscador/búsqueda de imágenes
- Interacción con contenido dentro de la plataforma
 - Guardar pin en tableros

El objetivo de estas pruebas fue verificar que el sistema respondiera correctamente ante distintos escenarios, tanto positivos como negativos, de todas las posibles interacciones con la plataforma de Pinterest.

Diseño y ejecución de pruebas

Para validar las funcionalidades del sistema se diseñaron más de **30 casos de prueba funcionales**, documentando para cada uno:

- Identificador del caso de prueba
- Descripción
- Precondiciones
- Pasos a ejecutar
- Resultado esperado
- Resultado obtenido

Estos casos de prueba tienen como fin evaluar el comportamiento del sistema y, asimismo, registrar el estado de cada ejecución realizada.

Listado de Bugs

ID	Título	Pasos para reproducir	Resultado esperado	Resultado obtenido	Severidad
BUG01	Registro permite enviar formulario sin email	1. Ir al formulario de registro 2. Dejar el campo de email vacío 3. Completar password 4. Presionar registrar	El sistema debe mostrar mensaje indicando que el email es obligatorio	El sistema intenta procesar el registro	Alta
BUG02	Validación de password débil no se aplica	1. Ir al registro 2. Ingresar password de menos de	El sistema debe rechazar el password y mostrar	El sistema permite continuar	Media

	correctamente	6 caracteres 3. Enviar formulario	error		
BUG03	Login acepta credenciales inválidas sin mostrar mensaje claro	1. Ir al login 2. Ingresar password incorrecto 3. Presionar iniciar sesión	El sistema debe mostrar mensaje de credenciales incorrectas	El sistema no muestra mensaje claro	Alta
BUG04	Campo password vacío no muestra advertencia inmediata	1. Ir a login 2. Dejar campo password vacío 3. Presionar iniciar sesión	El sistema debe mostrar mensaje de campo obligatorio	El sistema intenta procesar el login	Media
BUG05	Búsqueda sin resultados no muestra mensaje adecuado	1. Iniciar sesión 2. Buscar una palabra inexistente	El sistema debe mostrar mensaje "sin resultados"	La página queda en blanco o sin feedback	Baja
BUG06	Permite guardar pines duplicados sin advertencia	1. Abrir un pin 2. Guardarlo en un tablero 3. Guardar el mismo pin nuevamente	El sistema debería advertir que el pin ya está guardado	El pin se guarda duplicado	Baja
BUG07	Creación de tablero sin nombre permitido	1. Ir a crear tablero 2. Dejar campo nombre vacío 3. Guardar tablero	El sistema debe mostrar error de campo obligatorio	El tablero se crea sin nombre	Media
BUG08	Edición de tablero no guarda cambios correctamente	1. Editar nombre del tablero 2. Guardar cambios	El nombre del tablero debe actualizarse	El nombre permanece igual	Media
BUG09	Eliminación de tablero no solicita confirmación	1. Seleccionar tablero 2. Presionar eliminar	El sistema debería pedir confirmación antes de eliminar	El tablero se elimina inmediatamente	Baja
BUG10	Seguimiento de usuario no actualiza estado visual	1. Ir al perfil de un usuario 2. Presionar "seguir"	El botón debe cambiar a "siguiendo"	El botón permanece igual	Baja

→ Bugs en severidad **media** 4

→ Bugs en severidad **alta** 2

→ Bugs en severidad **baja** 4

Total de bugs encontrados: 10

40:40:20%

Recomendaciones

Los bugs encontrados demuestran que la mayoría de los problemas se concentran en la etapa de validación de entrada de datos y en el control de acciones del usuario, para ser más precisos en los módulos de registro, autenticación y gestión de contenido, donde se detectaron defectos importantes como:

→ Permitir el envío del formulario sin registrar un email.

- No detonar una recomendación o alerta al detectar contraseñas débiles.
- Criterio de aceptación erróneo de credenciales inválidas en el login.
- Permitir crear tableros sin nombre o con emojis únicamente.
- Ausencia de criterio de confirmación para eliminar tableros.

Se resalta que estos problemas pueden resultar relevantes, ya que pueden ser causantes de afectar la experiencia del usuario y la usabilidad de la plataforma y, de manera especial, la integridad de los datos.

En general, el sistema presenta una estructura funcional, pero es notorio que requiere de mejoras relevantes en los controles de validación, así como en el manejo de errores. Por eso se enlistan las siguientes recomendaciones y áreas de mejora para la plataforma:

1. Fortalecer las validaciones de formularios

Implementar validaciones de carácter obligatorio en campos críticos como correo electrónico, contraseña y nombre de tablero que contenga caracteres alfanuméricos, con el fin de evitar que el sistema procese datos incompletos o inválidos.

2. Mejorar los mensajes de error al usuario

Los mensajes de error deben ser claros y visibles para que el usuario entienda qué está ocurriendo, qué lo está detonando y, por supuesto, cómo corregirlo.

3. Implementar controles de duplicidad

Evitar que un mismo pin pueda guardarse varias veces en el mismo tablero sin advertencia al usuario.

4. Agregar una doble validación/confirmación en acciones críticas

En las acciones que resultan relevantes para el uso de la cuenta, tales como eliminar tableros, pines o incluso la cuenta, deben solicitar confirmación previa con un pop-up o botón de confirmación para evitar pérdidas accidentales de información.

5. Mejorar el feedback visual en la UI

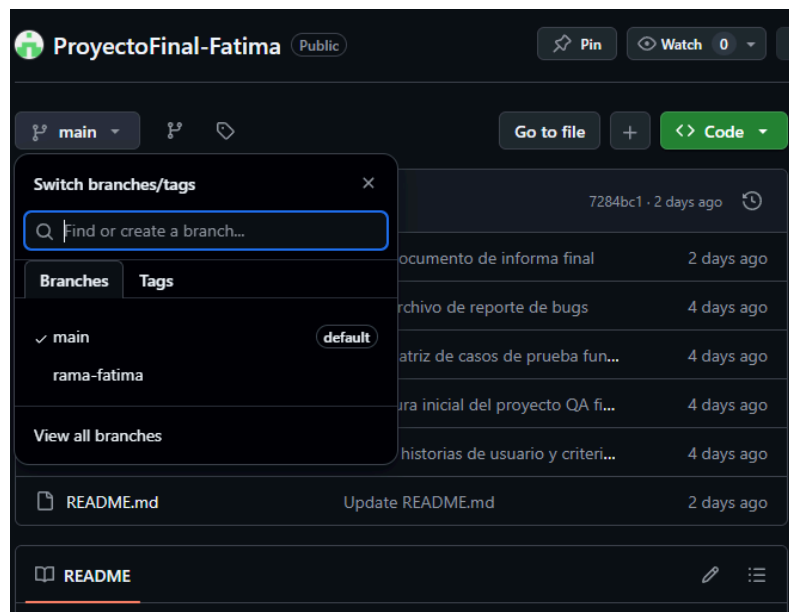
Para mejorar la experiencia del usuario y haciendo uso de la paleta de colores de la plataforma, se deberán implementar y reflejar cambios visuales inmediatos al realizar determinada acción como *follow/unfollow*, *pinned/unpinned*, *guardar/guardado*.

Uso de Git y GitHub

Para la gestión del proyecto y tener el control de versiones, se utilizó la herramienta de **GitHub**, que me permitió organizar el trabajo mediante **commits** descriptivos y mantener una estructura del repositorio registrando de manera correcta, puntual y precisa toda la documentación del proceso de testing.

Ramas y Commits

Main: y rama propia:



Windows PowerShell

Durante el uso de Windows PowerShell junto con Git pudimos gestionar el repositorio del proyecto para mantener organizados todos los archivos, documentación, evidencias y pruebas.

El uso de la terminal para el control de las versiones y poder subir los avances a GitHub fue a través de diversos comandos, de entre los cuales se destacan *git status* para conocer el estado actual del repositorio, *git add* . para cuando era necesario añadir los archivos añadidos sin rastreo aún, *git commit* para registrar los cambios, y *git push* para subirlos de forma remota al repositorio.

En momentos, trabajamos con dos ramas, que son *main* y *rama-fátima*, lo que permitió que se pudiera trabajar y organizar los cambios sin afectar directamente a la rama principal.

Uno de los desafíos que encontramos fue cuando Git rechazó un push porque el repositorio remoto tenía cambios que nuestra versión local no tenía. Para resolverlo, utilizamos *git pull* para integrar los cambios y posteriormente realizar el push nuevamente.

```
Windows PowerShell
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  evidencias/

no changes added to commit (use "git add" and/or "git commit -a")
PS C:\ProyectoFinal-Fatima> git add .
PS C:\ProyectoFinal-Fatima> git commit -m "moví la carpeta de screenshots a la correcta que es evidencias"
[rama-fatima 9213b91] moví la carpeta de screenshots a la correcta que es evidencias
4 files changed, 0 insertions(+), 0 deletions(-)
 rename {api_testing => evidencias}/Capturas/api_get_post.png (100%)
 rename {api_testing => evidencias}/Capturas/api_get_post1.png (100%)
 rename {api_testing => evidencias}/Capturas/api_negative_404.png (100%)
 rename {api_testing => evidencias}/Capturas/api_post_create.png (100%)
PS C:\ProyectoFinal-Fatima> git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 404 bytes | 202.00 KiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/falazarin/ProyectoFinal-Fatima.git
  79157cf..9213b91  rama-fatima -> rama-fatima
PS C:\ProyectoFinal-Fatima> git status
On branch rama-fatima
Your branch is up to date with 'origin/rama-fatima'.

nothing to commit, working tree clean
PS C:\ProyectoFinal-Fatima> |
```

Postman

ID	Método	Endpoint	Resultado
API01	GET	/posts	200 OK
API02	GET	/posts/1	200 OK
API03	GET	/posts/9999	404 Not Found
API04	POST	/posts	201 Created
API05	PUT	/posts/1	200 OK
API06	GET	/posts/1	validación JSON
API07	GET	/comments	200 OK
API08	GET	/comments/1	200 OK
API09	GET	/users	200 OK
API10	GET	/users/9999	404 Not Found

ID	Método	
API01	Get ▾	https://jsonplaceholder.typicode.com/posts
API02	Get ▾	https://jsonplaceholder.typicode.com/posts/1
API03	Get ▾	https://jsonplaceholder.typicode.com/posts/9999
API04	Get ▾	https://jsonplaceholder.typicode.com/posts
API05	Get ▾	https://jsonplaceholder.typicode.com/posts/1
API06	Put ▾	https://jsonplaceholder.typicode.com/posts/1
API07	Get ▾	https://jsonplaceholder.typicode.com/comments
API08	Get ▾	https://jsonplaceholder.typicode.com/comments/1
API09	Get ▾	https://jsonplaceholder.typicode.com/users
API10	Get ▾	https://jsonplaceholder.typicode.com/users/9999

Reporte de defectos

Durante el proceso de testing se están documentando los defectos simulados con el objetivo de representar el flujo de reporte de bugs de forma similar a como sucedería dentro de un proyecto real.

Cada defecto fue documentado incluyendo:

- El ID del bug
 - Los pasos para reproducir el error
 - El resultado esperado y el resultado obtenido
 - La severidad del defect
-

Testing de la API

También se realizaron pruebas de la API utilizando **Postman**, donde se utilizaron endpoints de la API pública **JSONPlaceholder** para validar distintos tipos de solicitudes HTTP, incluyendo:

- Las consultas GET
- La creación de recursos mediante POST
- Una actualización de recursos mediante PUT
- La validación de estructura de respuestas JSON
- Las pruebas negativas sobre endpoints inexistentes

En total, se ejecutaron **10 casos de prueba** de la API y se verificaron los códigos de respuesta (200, 201), así como el contenido de las respuestas y el comportamiento ante errores (404).

Conclusión

El proceso de testing nos ha permitido identificar los aspectos a mejorar dentro de Pinterest Lite, resaltando la importancia de que las pruebas sistemáticas durante el desarrollo de un sistema sean tan relevantes para el éxito y buen funcionamiento de cualquier app y/o sistema. Esta documentación tiene el fin de que se evalúe el estado del desarrollo así como el comportamiento de la aplicación dentro de un alcance definido para este proyecto.

Contar con herramientas como Git y GitHub gestionadas de forma remota desde PowerShell resultó ser de gran beneficio para poder mantener un orden y una organización del repositorio del proyecto, así como poder registrar minuciosamente cualquier cambio y asegurarnos de que toda la documentación y evidencias del proceso de pruebas estén correctamente almacenadas y disponibles.

Finalmente, la implementación de las recomendaciones propuestas ayudaría a reducir errores funcionales, así como a optimizar la interacción del usuario con la plataforma y a asegurar un comportamiento más consistente del sistema que contribuya así a la percepción del usuario de una aplicación más confiable y, por ende, a una mejor experiencia del usuario final.